



Marco Faella

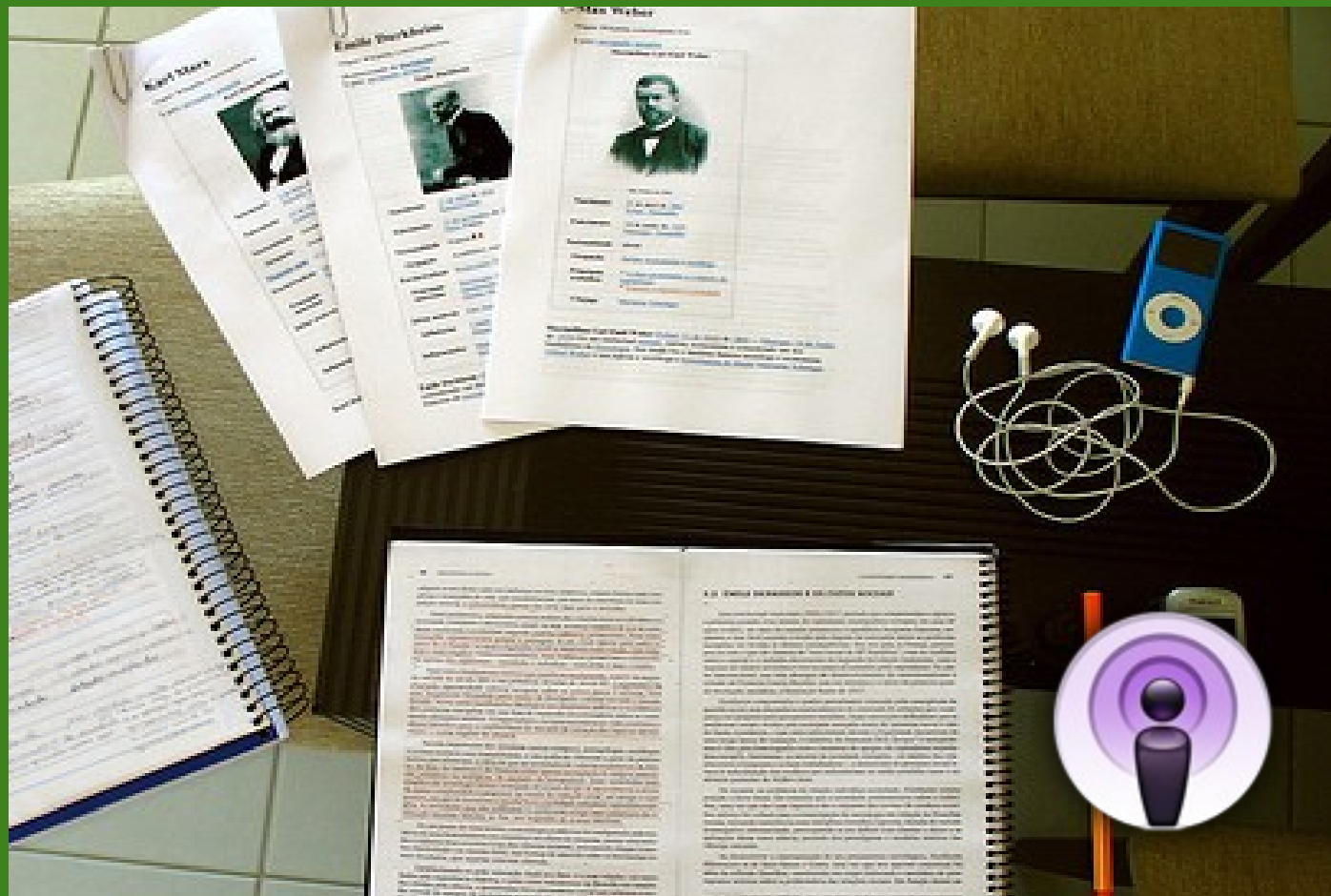
Introduzione al corso

Lezione n. 1
Parole chiave:
Java

Corso di Laurea:
Informatica

Insegnamento:
Linguaggi di Programmazione II

Email Docente:
faella.didattica@gmail.com



Testi consigliati:

- Programmazione Java, di Claudio de Sio Cesari, Hoepli 2025 (Java 25)

Alternative:

- Core Java, volumi 1 e 2, di Cay Horstmann (almeno 13^a edizione, Java 21)
- Java Precisely (3^a edizione, Java 8), di Peter Sestoft, MIT Press

Inoltre:

- Seriously Good Software, di M.F., Manning 2020

Per l'approfondimento:

- Linguaggi di programmazione, principi e paradigmi
di M. Gabbrielli e S. Martini
McGraw-Hill

Il capitolo 10, in particolare, offre un'introduzione rigorosa al polimorfismo parametrico (*generics*)

- Questo corso presuppone la conoscenza dei seguenti argomenti di Java elementare:
 - Tipi base e regole di compatibilità
 - Operatori ed espressioni
 - Istruzioni di controllo di flusso (if-then-else, for, while, break, continue)
 - Classi
 - Costruttori, metodi, campi
 - Modificatori di visibilità
 - Modificatori static e final
 - Interfacce
 - Ereditarietà
 - Classi astratte
 - Eccezioni
 - Classi interne
 - Differenza eccezioni verificate (*checked*) e non (*unchecked*)

Vero o falso?

- 1) Una variabile locale di un metodo può essere dichiarata `static`.
- 2) Un costruttore può essere privato.
- 3) Un'interfaccia può avere un costruttore.
- 4) Una classe deve necessariamente implementare un costruttore.
- 5) Tutte le classi derivano da (cioè sono sottoclassi di) `Class`.
- 6) Tutte le eccezioni derivano da `Exception`.
- 7) “`float f = 2.0`” è un'istruzione corretta.
- 8) Se `x` è un riferimento ad una stringa, l'istruzione “`x instanceof Object`” restituisce `false`.
- 9) Un'interfaccia può avere metodi statici.
- 10) Una variabile locale di un metodo può essere dichiarata `private`.

Trovare gli errori

se ce ne sono...

```
public class test {  
    private int x, y;  
    private test other;  
    int f(void)    { return 3; }  
    long f(void)  { return 3L; }  
  
    public static main(String[] args) {  
        int i;  
        int j[10];  
        String a = "ciao";  
        j[0] = i;  
  
        test t = new test();  
        long l = t.f();  
  
        return 0;  
    }  
}
```

- Si utilizzerà talvolta la nozione di “**memory layout**” di un programma
- Ci si riferisce ad una rappresentazione grafica dello stato della memoria ad un determinato punto di un programma
- Come primo esempio, dato il frammento di codice Java:

```
class A {
    private String x;
    private int n;
    public A(String x, int n) {
        this.x = x;
        this.n = n;
    }
}
...
A a = new A("ciao", 7);
```

- Il suo memory layout può essere raffigurato come in Figura 1
- I diagrammi di memory layout di queste slide non evidenziano la differenza tra allocazione su stack e su heap

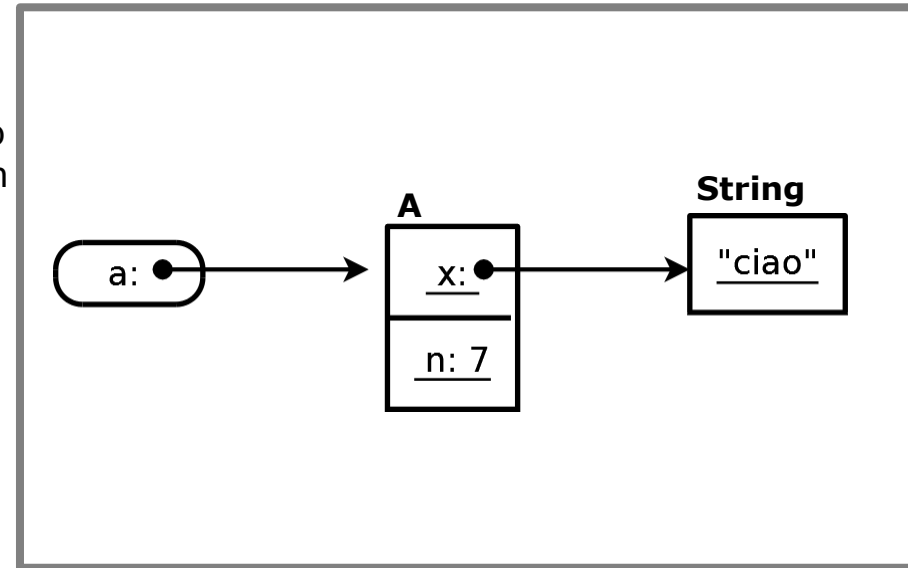


Figura 1: Il memory layout del frammento di programma a fianco.

- In Java, gli array sono oggetti a tutti gli effetti
- In particolare, sono sottotipi di Object e ne ereditano i metodi
- Non esistono variabili che contengono (direttamente) array, ma solo riferimenti ad array
- Ad esempio, in figura viene riportato il memory layout del seguente frammento di programma:

```
int[] a = new int[5];
int[] b = a;
int[][] c = new int[3][];
c[1] = b;
c[2] = new int[7];
```

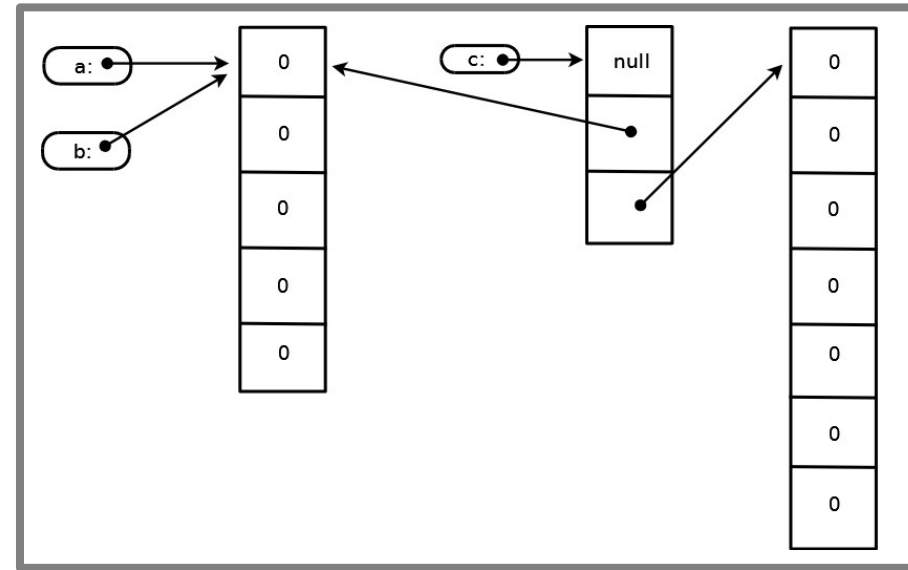


Figura 2: Il memory layout del frammento di programma a fianco.